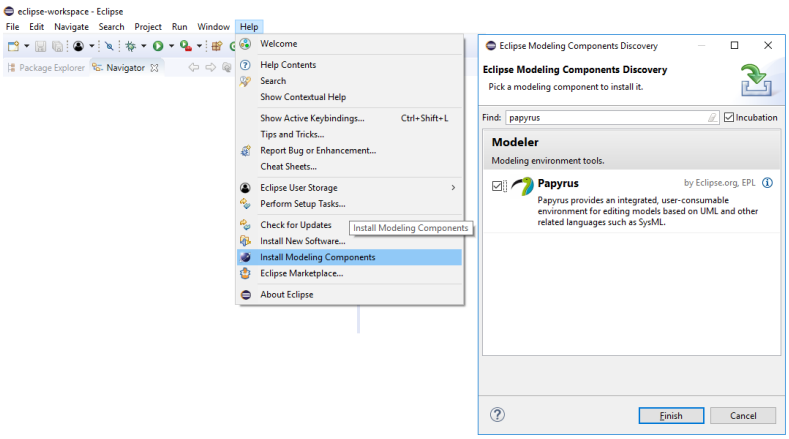


Figure 3 Papyrus installation

At this point you have successfully completed the environment setup. All the required software is installed and you are ready to start using **B4MSecure**. The next step is to create an UML model describing the functional and security aspects of your system.

2 System Modeling

We will use **Papyrus** to create the models used by B4MSecure. We expect the reader to be familiar with the UML class diagram editor, otherwise please refer to a Papyrus tutorial. Papyrus documentation is available at https://www.eclipse.org/papyrus/documentation.html

Use the new project wizard ("File" > "New" > "Project") to create a new Papyrus project named "*HospitalSystem"*, with a root Class Diagram.

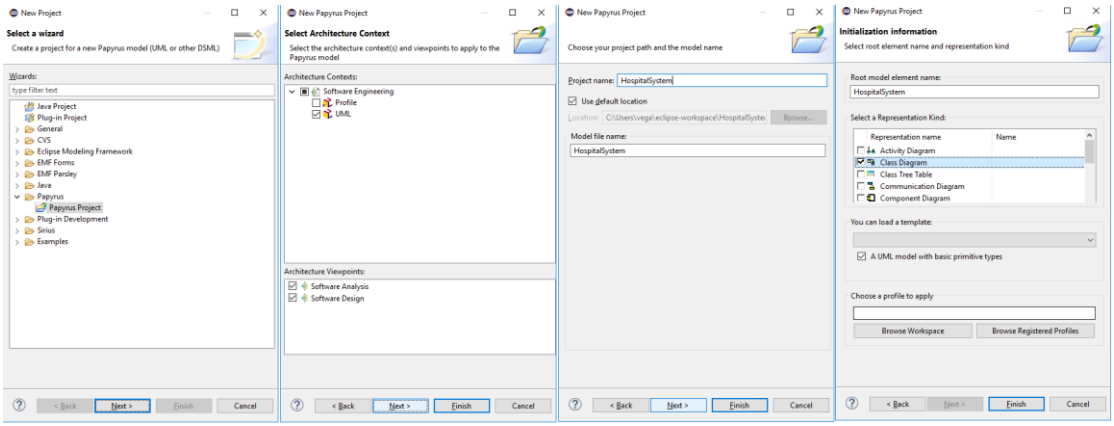


Figure 4 Papyrus project's creation

If you go to the Papyrus perspective, you should see a project named "*HospitalSystem*", with a single empty UML model file (named "*HospitalSystem"* as well). We are going now to create the functional and access policy packages.

2.1 Structuring the model

First, use the *palette* of the root class diagram to create two packages: the first one will store the functional model, and the second one will store the security model. Let us name these packages "***Functional***" and "***Policy***". Use the *Model Explorer* view to create a class diagram for each package, as shown in the following figure.

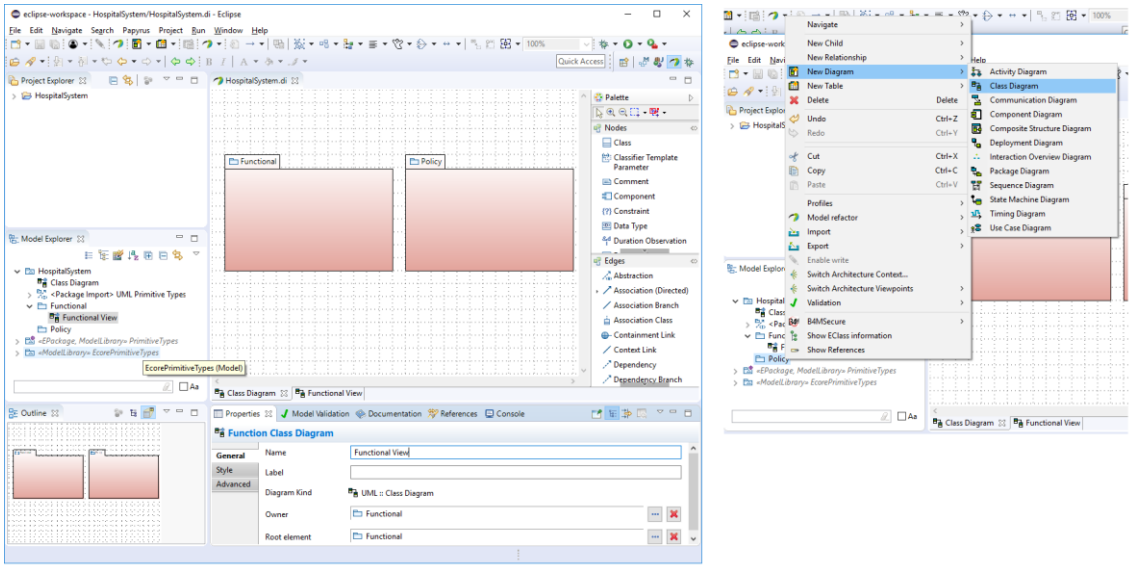


Figure 5 Structuring the model

2.2 Creating the functional model

Select the class diagram of the "***Functional***" package, and build the following model:

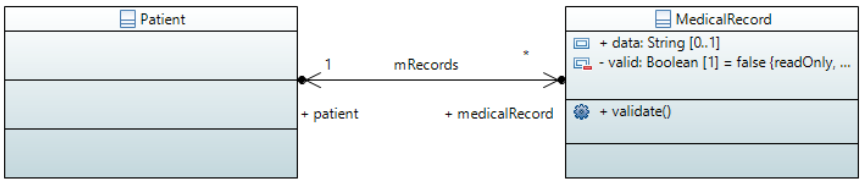


Figure 6 Hospital System, functional model

Attribute "***data***" is declared *public* and *optional*[3]. Attribute "***valid***" is *private*, *mandatory* and has a *default value* of *false*; it is also declared as *read only*, this means that no modification operation will be generated in the B specification for this attribute. Association "***mRecords***" links medical records to a single patient, while a patient may have several medical records. This association can be *navigated* in both directions. All these features are specified in the *Properties* view of the editor. Method "***validate***" is a *public* operation of class "***MedicalRecord***", with no parameters or result.

[3] *optional* attributes have multiplicity **0..1**, *mandatory* attributes have multiplicity **1**

2.3 Generation of the B functional specification

At this point, we have a functional model of the system; we can now use **B4MSecure** to generate the corresponding B specification. **B4MSecure** is available in the context menu associated to an UML package (it is activated by selecting a package and right-clicking on it).

In our scenario, we are going to select the "***Functional***" package in the *Model Explorer*, open the contextual menu and choose the B4MSecure transformation, as shown in the following figure.

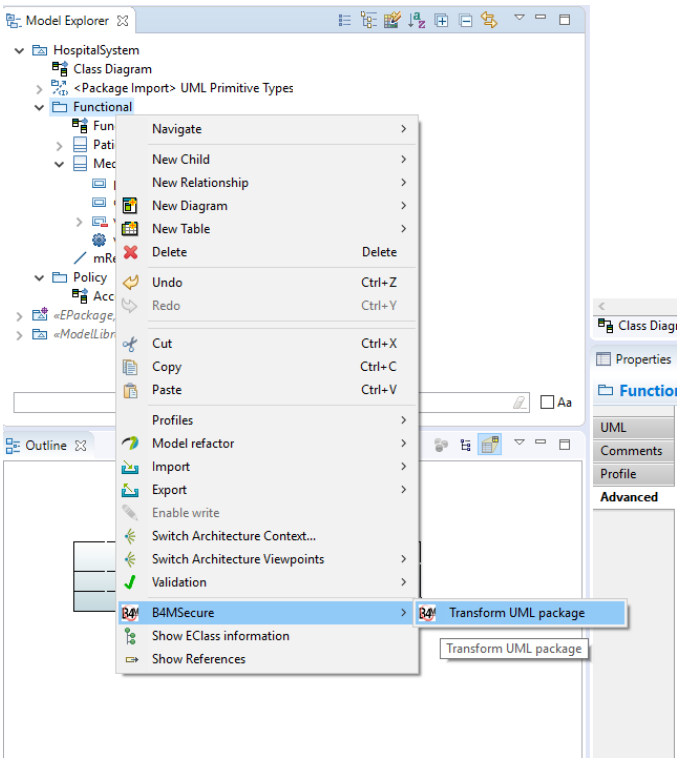


Figure 7 Generation of the B specification using B4MSecure

After execution of the B4mSecure transformation[4], several files are created in the same folder of the UML model, as shown in Figure 8. The generated **B specification machine** has the same name of the selected package, with the file extension "mch". So, in our case it is named "*Functional.mch*" (we can ignore, by now, the other generated B machine files, which are related to the security policy).

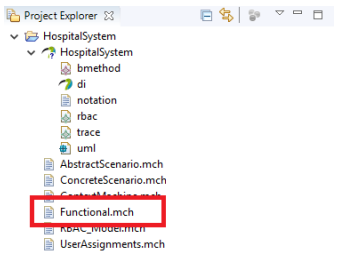


Figure 8 B4MSecure generated files

[4] You can see the log of the execution of the transformation in the *Console* view

In the resulting B specification, shown in Figure 9, classes are represented by sets (e.g. *PATIENT* and *MEDICALRECORD*). The existing instances of the class are represented by variables (e.g. *Patient* and *MedicalRecord*). Attributes are translated to functions (e.g. *MedicalRecord_data* and *MedicalRecord_valid* [5]). Associations are represented as a relation between the sets of instances (e.g. *mRecords*[6]).

HospitalSystem.di Functional.mch

```
1 MACHINE
2     Functional
3
4 SETS
5     STR;
6     PATIENT;
7     MEDICALRECORD
8
9 ABSTRACT_VARIABLES
10    Patient,
11    MedicalRecord,
12    mRecords,
13    MedicalRecord_data,
14    MedicalRecord_valid
15
16 INVARIANT
17    Patient : FIN(PATIENT) &
18    MedicalRecord : FIN(MEDICALRECORD) &
19    mRecords : MedicalRecord --> Patient &
20    MedicalRecord_data : MedicalRecord +-> STR &
21    MedicalRecord_valid : MedicalRecord --> BOOL
22
23 INITIALISATION
24    Patient := {} ||
25    MedicalRecord := {} ||
26    mRecords := {} ||
27    MedicalRecord_data := {} ||
28    MedicalRecord_valid := {}
29
30 OPERATIONS
31    MedicalRecord_validate(aMedicalRecord) =
32    PRE aMedicalRecord : MedicalRecord
33
34    THEN skip
35    END;
36
37    Patient_NEW(aPatient) =
38    PRE aPatient : PATIENT &
39        aPatient /: Patient
40
41    THEN Patient := Patient \/ {aPatient}
42    END;
```

Figure 9 B functional specification

Notice also that each method defined in a class in the model is translated to a B operation (e.g. *MedicalRecord_validate*), that takes as parameter an instance of the class. The generated operation has an empty (skip) substitution.

B4MSecure also generates basic operations to manipulate the model: class constructors (e.g. *Patient_NEW*), class destructors, attribute's accessors, attribute's setters, association navigation, and association's setters. A detailed explanation of the B4MSecure translation is beyond the scope of this manual; see the *publications* section of the site for a more thorough description.

[5] Notice that optional attributes, for example *data,* are translated to a partial function

[6] Notice that in this particular case, the *Patient* end of the association *mRecords* has multiplicity 1, so the relation is a total function

2.4 Adding invariants to the B specification

The generated B specification has invariants that essentially represent typing and multiplicities defined in the UML model. You may want to add domain invariants to your specification.

One possibility is to directly edit the generated file; however, your changes will be lost the next time that you execute the B4MSecure transformation. B4Msecure then offers the possibility to directly specify your B invariants in the UML model file.

Consider for instance the following invariant for the *Hospital System*: "a patient cannot have more than one medical record that is not validated". Given the generated B specification, this invariant can be expressed by the following B predicate:

$$!(p).(p : Patient \Rightarrow card((mRecords\sim[\{p\}] \lhd MedicalRecord_valid) \rhd \{FALSE\}) \leq 1)$$

To add this invariant to your UML model, select the "***Functional"*** package in the *Model Explorer*, and a new *Constraint*, as shown in the following figure:

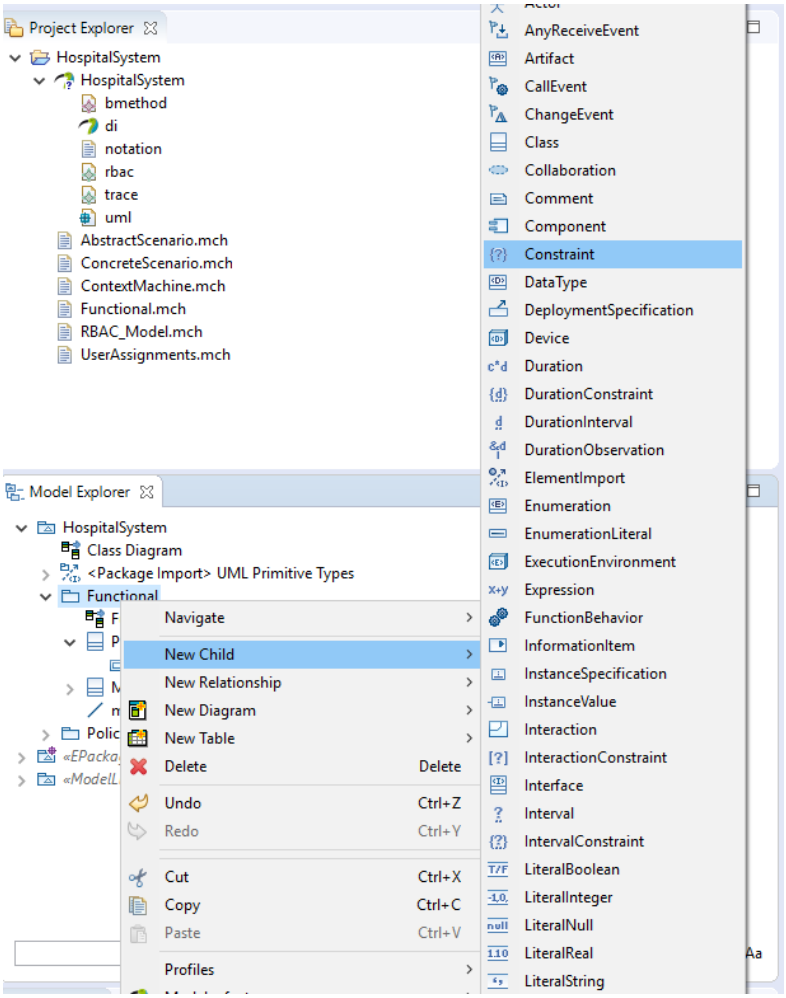


Figure 10 Adding constraints to the functional package

By default, Papyrus will create an OCL constraint, as shown in Figure 11; but it is possible to specify constraints in other languages. To add a new language: select the constraint value in the *Model Explorer*, go to the *Properties* view, click the "+" sign in the language section (red-circled in the

figure). You will be prompted to add the new language (as shown in the inset, at the right of the figure), add the **B** language.

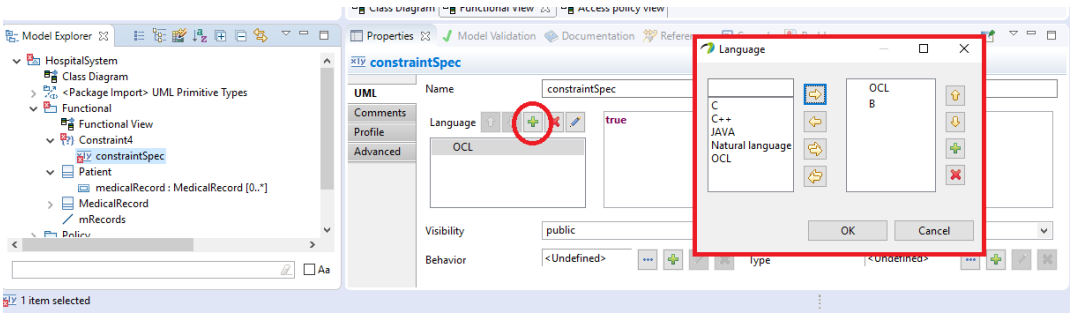


Figure 11 Papyrus: adding a new language to a constraint

Once you have added the B language, select it, and type your B predicate in the *Properties* view, as shown in Figure 12. Notice that we have also removed the unneeded OCL constraint, and renamed the constraint to give it a meaningful name.

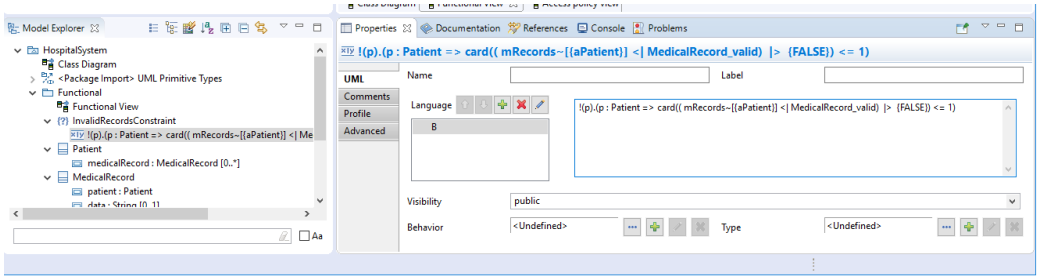


Figure 12 Specifying a B constraint to the model

At this point, the B invariant has been added to the UML model. We can now execute B4Msecure (see section 2.3) and regenerate the B specification. It will include the corresponding invariant.

*HospitalSystem.di | Functional.mch

```
1  MACHINE
2      Functional
3
4  SETS
5      STR;
6      PATIENT;
7      MEDICALRECORD
8
9  ABSTRACT_VARIABLES
10     Patient,
11     MedicalRecord,
12     mRecords,
13     MedicalRecord_data,
14     MedicalRecord_valid
15
16 INVARIANT
17     Patient : FIN(PATIENT) &
18     MedicalRecord : FIN(MEDICALRECORD) &
19     mRecords : MedicalRecord --> Patient &
20     MedicalRecord_data : MedicalRecord +-> STR &
21     MedicalRecord_valid : MedicalRecord --> BOOL &
22     !(p).(p : Patient => card(( mRecords~[{aPatient}] <| MedicalRecord_valid)  |>  {FALSE}) <= 1) /* Defined in UML
23
24 INITIALISATION
25     Patient := {} ||
26     MedicalRecord := {} ||
27     mRecords := {} ||
28     MedicalRecord_data := {} ||
29     MedicalRecord_valid := {}
30
31 OPERATIONS
```

Figure 13 Domain invariants in generated B specification

2.5 Adding preconditions and operation's body

As we pointed out, methods defined in the UML model are translated by default into an empty B operation skeleton. To complete the B specification, we may need to add preconditions and specify the behavior of these methods.

Consider for instance the method "***validate"*** in the *Hospital System*. We can add a simple pre-condition to express the fact that validation only occurs for invalid records. This can be expressed in the B specification as predicate over the parameter of the operation *MedicalRecord_validate*:

MedicalRecord_valid(aMedicalRecord) = FALSE

In a similar way, one effect of the operation is to record the validation in the corresponding attribute. This can be expressed by the following B substitution:

MedicalRecord_valid(aMedicalRecord) := TRUE

As was the case for the invariants, we want to keep this information in the UML model (so that we do not need to edit the generated file). B4Msecure offers this possibility using the same approach that for invariants: we add constraints to the method in the UML model.

In our example, we select the "***validate"*** method in the *Model Explorer*, and add two B constraints (the procedure is the same as for the invariants, except that we select the method not the package), the result is shown in Figure 14.

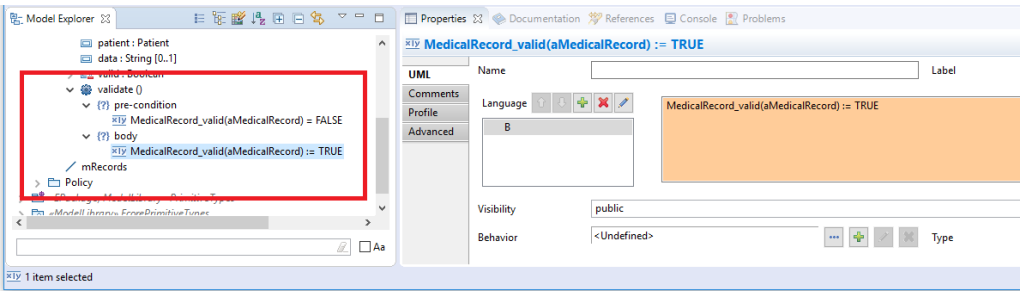


Figure 14 Defining pre-conditions and body

After that, select the "***validate"*** method, and in the *Properties* view change its body (as shown in the following figure) by selecting the constraint just defined.

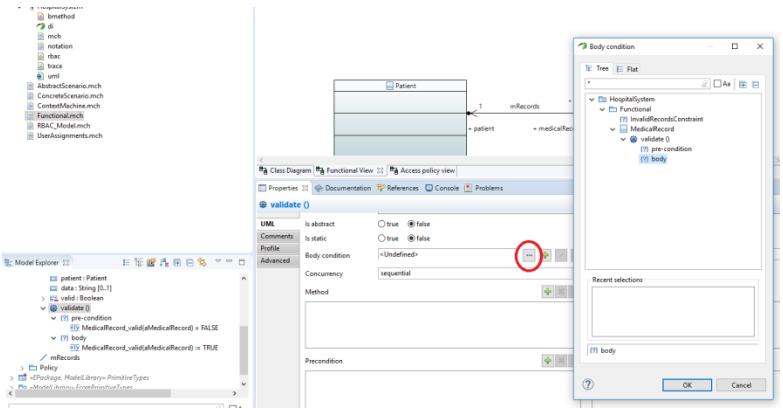


Figure 15 Specifying operation's body

In a similar way, go to the *precondition* section in the Properties view, and select the corresponding constraint, as shown in Figure 16.

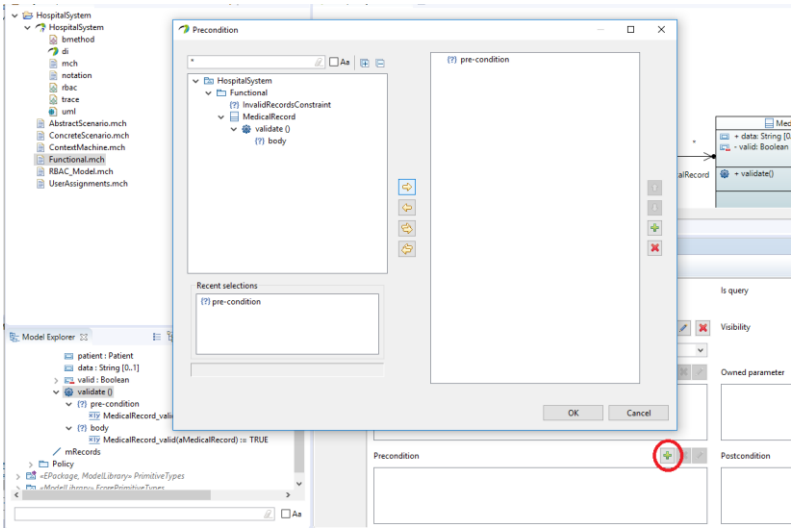


Figure 16 Specifying preconditions

At this point, the B precondition and body have been added to the UML model. We can now execute B4Msecure (see section 2.3) and regenerate the B specification.

HospitalSystem.di | Functional.mch

```
1  MACHINE
2      Functional
3
4  SETS
5      STR;
6      PATIENT;
7      MEDICALRECORD
8
9  ABSTRACT_VARIABLES
10     Patient,
11     MedicalRecord,
12     mRecords,
13     MedicalRecord_data,
14     MedicalRecord_valid
15
16 INVARIANT
17     Patient : FIN(PATIENT) &
18     MedicalRecord : FIN(MEDICALRECORD) &
19     mRecords : MedicalRecord --> Patient &
20     MedicalRecord_data : MedicalRecord +-> STR &
21     MedicalRecord_valid : MedicalRecord --> BOOL &
22     !(p).(p : Patient => card(( mRecords~[{aPatient}] <| MedicalRecord_valid)  |>  {FALSE}) <= 1) /* Defined in UML
23
24 INITIALISATION
25     Patient := {} ||
26     MedicalRecord := {} ||
27     mRecords := {} ||
28     MedicalRecord_data := {} ||
29     MedicalRecord_valid := {}
30
31 OPERATIONS
32     MedicalRecord_validate(aMedicalRecord) =
33     PRE aMedicalRecord : MedicalRecord &
34         /* Defined in UML model  */ MedicalRecord_valid(aMedicalRecord) = FALSE
35
36     THEN /* Defined in UML model  */ MedicalRecord_valid(aMedicalRecord) := TRUE
37     END;
38
39     Patient_NEW(aPatient) =
40     PRE aPatient : PATIENT &
41         aPatient /: Patient
42
43     THEN Patient := Patient \/ {aPatient}
```

Figure 17 Body and pre-conditions in generated B specification

We have finished our functional system model. Now we can start defining the associated security policy.

3 Security Modeling

To model the access control policy, B4MSecure uses a **UML profile** for RBAC (Role Based Access Control) that is inspired by **SecureUML**. In this section we will present how to use the profile in Papyrus, to define the security policy. We do not discuss in depth the concepts of SecureUML, the interested reader can refer to the paper [SecureUML: A UML-Based Modeling Language for Model-Driven Security] by Torsten Lodderstedt, David Basin, and Jürgen Doser.

3.1 Applying the SecureUML profile

The first step to use an UML profile is to ***apply* the profile** to your model. In Papyrus, this is done by selecting the *root* of your model, and using the *Profile* tab of the *Properties* view, as shown in Figure 18. To select the profile, click on the "*apply registered profile*" button (the *plug* icon, red-circled in the figure) and choose the B4MSecure - SecureUML profile.

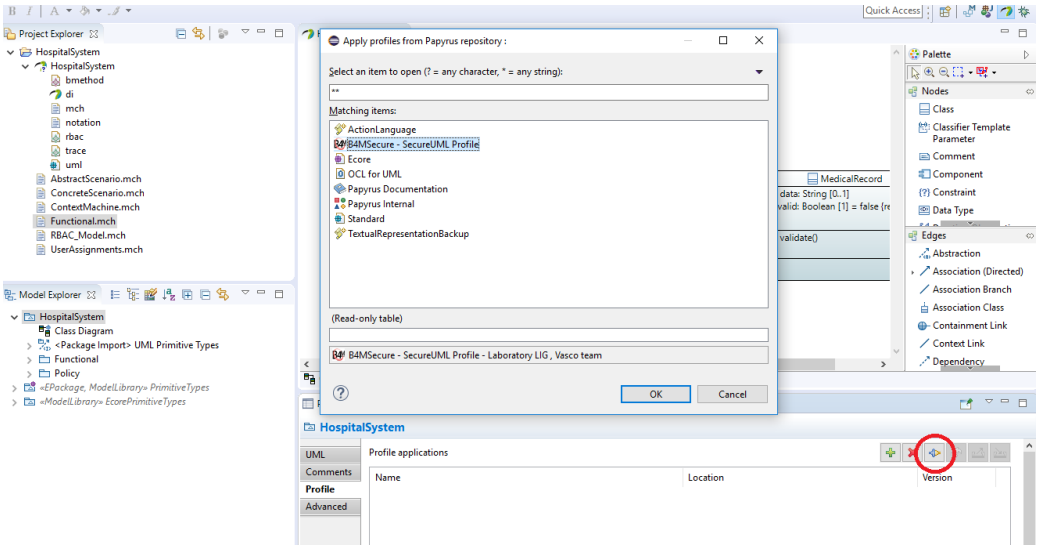


Figure 18 Applying the SecureUML profile

3.2 Defining the Role model

In SecureUML, security *roles* are represented by classes, and the *role hierarchy* is represented using a class generalization hierarchy. In our scenario, we are going to use the following role model:

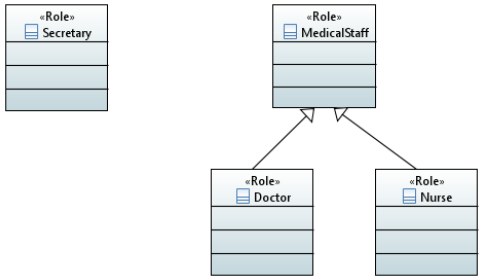


Figure 19 Role model (*Hospital System*)

To create the model open the class diagram associated to the "***Policy***" package, then create the classes and inheritance hierarchy shown in the previous figure. To differentiate domain classes from security roles, the SecureUML profile defines a **stereotype** named *Role*.

We need then to ***apply* the stereotype** to the classes in our UML model that represent roles. In Papyrus, this is done by selecting the class, and using the *Profile* tab of the *Properties* view, as shown in Figure 20. To select the stereotype, click on the "+" button (red-circled in the figure) and choose the *Role* stereotype.

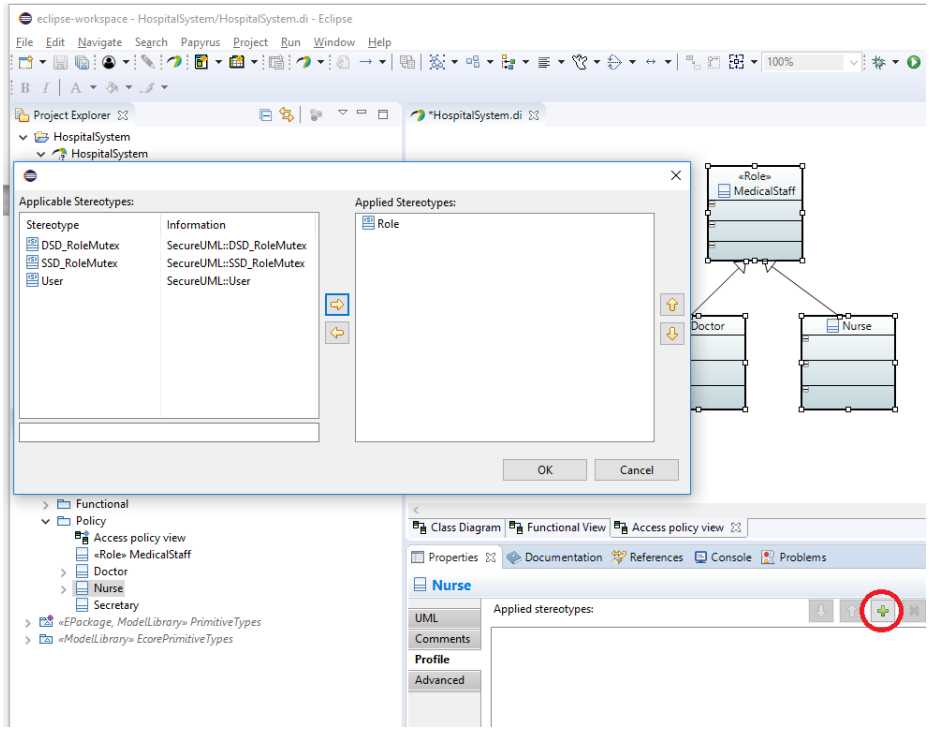


Figure 20 Applying an stereotype

At this point, we have finished defining the role model. We can now execute B4Msecure (see section 2.3) and regenerate the B specification. The corresponding B security specification is generated in file "*UserAssignments.mch".* A detailed explanation of the B4MSecure security translation is beyond the scope of this manual; but you can have a glimpse of the generated machine in Figure 21.

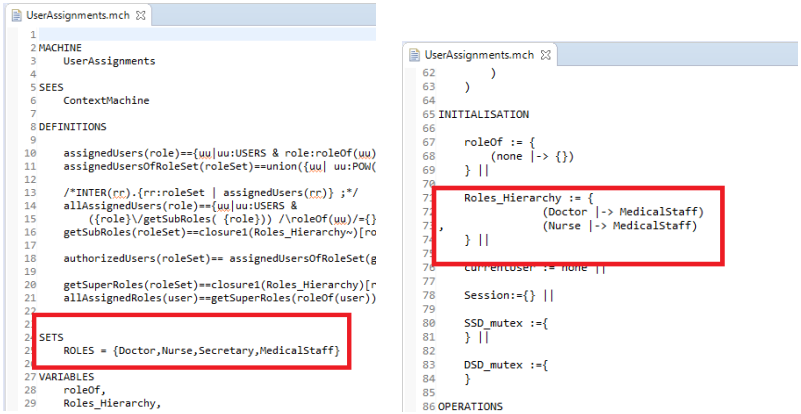


Figure 21 Role model in B

3.3 Defining the User Assignment model

In SecureUML, security *users* are represented by classes distinguished with the stereotype *User*. A *user assignment* (assigning a *role* to a *user*) is represented by defining an association between the class representing the *user* and the class representing the *role*.

In our scenario, we are going to use the following user assignment model:

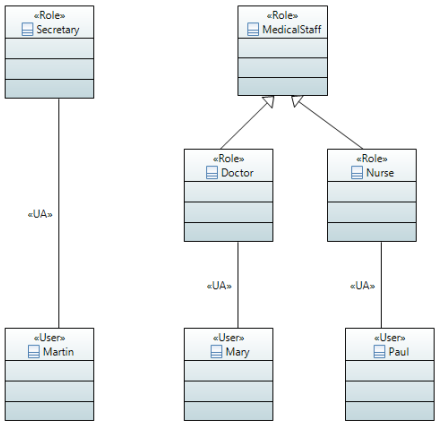


Figure 22 User Assignment

To create the model in Papyrus, just follow the same procedure explained in the previous section, and apply the stereotypes as appropriated. Notice that the associations have the applied stereotype *UA*. The procedure to apply a stereotype to an association is similar, just select the association and use the *Profile* tab in the *Properties* view.

Notice that Papyrus by default doesn't display the stereotypes of associations (and always displays ends' names and cardinalities). You can customize the display of an association in the diagram by selecting it and using the contextual (right-click) menu "Filters", as shown in the following figure.

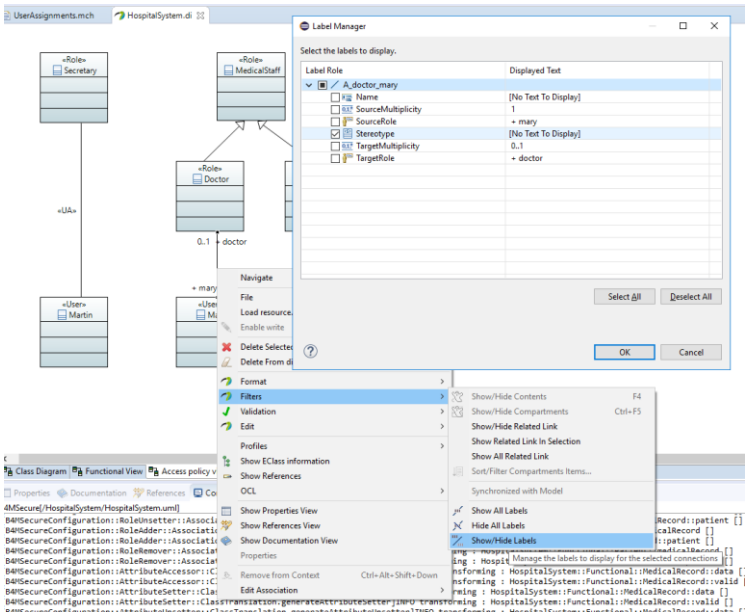


Figure 23 Customizing associations 'display

At this point, we have finished defining the user assignment model. We can now execute B4Msecure (see section 2.3) and regenerate the B specification. The corresponding B security specification is generated in files "*UserAssignments.mch"*.and "*ContextMachine.mch"* as shown in the following figure.

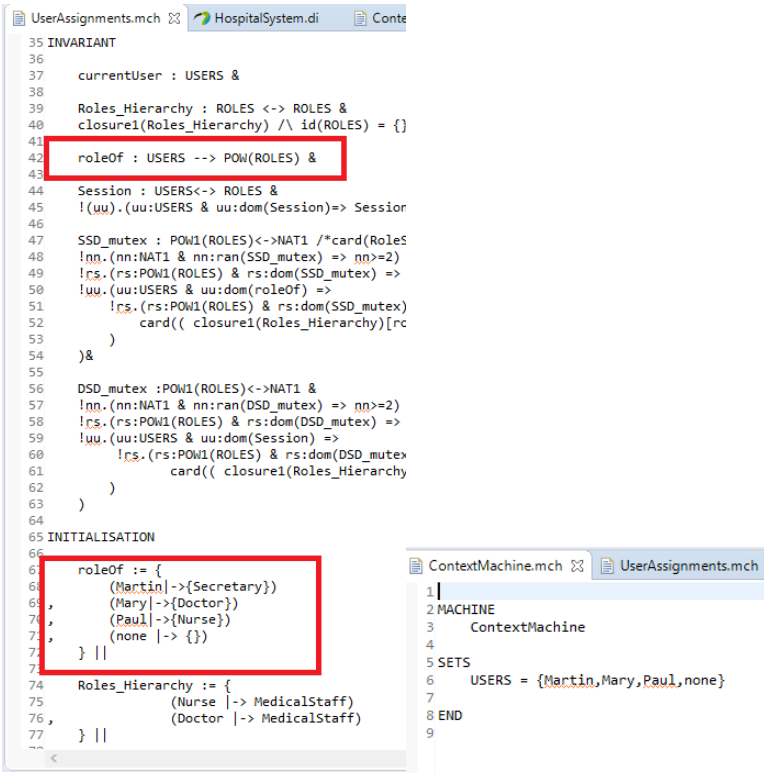


Figure 24 User asigment model in B

3.4 Defining the Permission model

In SecureUML, security *permissions* are represented by an association class between the class representing the *role* and the domain class representing the secured entity. Allowed actions are represented by methods of the permission.

For example, Figure 25 shows a permission policy expressing that a "*secretary may create a patient object"*. Notice the *Permission* stereotype applied to the association class, and the *EntityAction* stereotype applied to the allowed action *create*[7].

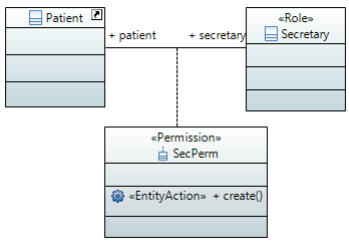


Figure 25 Secretary's permission

[7] The allowed entity actions that can be specified in SecureUML are : create, read, update/modify, delete

Figure 26 shows a policy expressing the following access rules: "*Every member of the medical staff may read the public information of medical record (this means that nurses and doctors have read access to the "Data" attribute)*". "*Nurses may only create a medical record*". "*Doctors may modify the data of a medical act and validate it".* Notice the *MethodAction* stereotype in operation ***validate***[8], it is used to express that the method can be executed by a user assigned to the specified role.

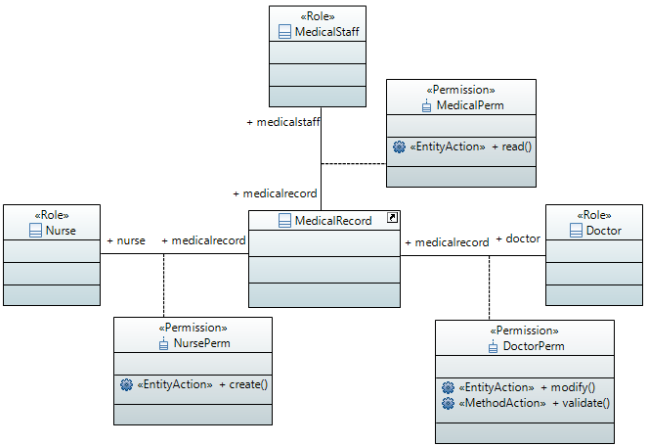


Figure 26 Security permission model

At this point, we have finished defining the *permission* model. We can now execute B4Msecure (see section 2.3) and regenerate the B specification. The corresponding B security specification is generated in file "*RBAC_model.mch"*, as shown in the following figure.

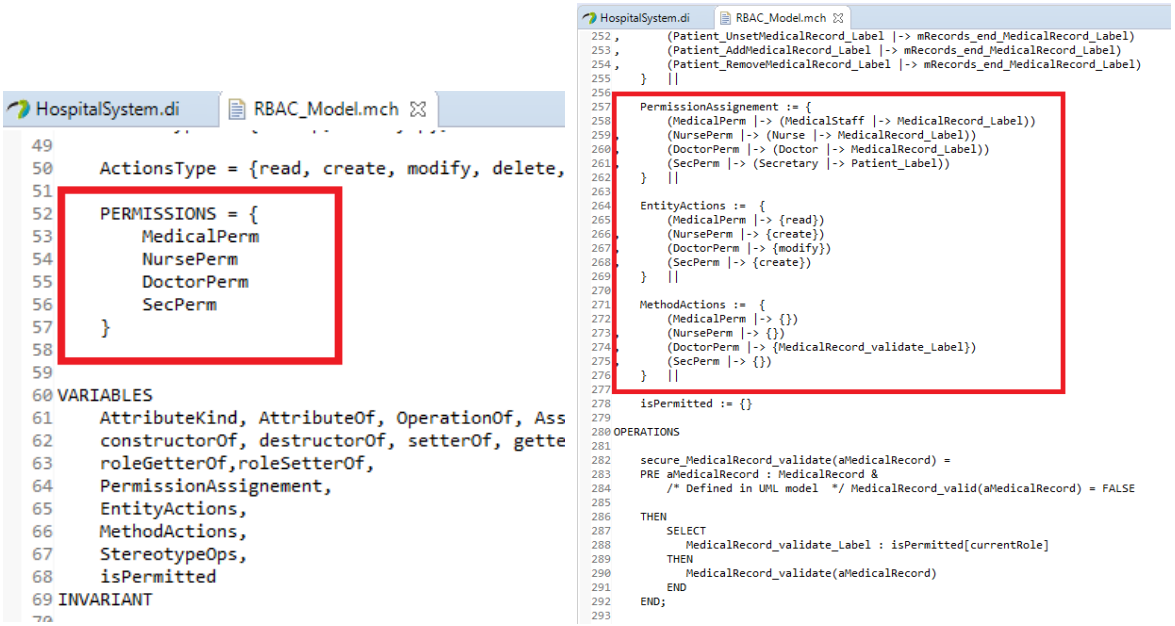


Figure 27 Permission specification model in B

[8] The name of the *Entity Action* method must match a method in the secured class

4 Conclusion

This guide has provided you with step-by-step instructions on how to use **Papyrus** and **SecureUML** to model secured systems in UML. Once you have completed your model, we have shown you how to use **B4MSEcure** to generate the corresponding B specification.

B4MSecure enables you to perform formal reasoning on your model. You can, for instance, take the B specification generated by **B4MSecure** and use the available tools of the B Method ecosystem for verifying proof obligations, or perform model checking.

We hope that **B4MSecure** will be a useful tool for you, but please contact us if you experience problems or simply want to request an enhancement.